Learning to Rank

Avishek Anand

Overview

- What is ranking?
- Why is LTR hard?
- Pointwise methods
- Pairwise methods
- Listwise methods
- Metric-based methods
- Stochastic approaches

The Ranking Pipeline

Scenario 1: Product Search (E-commerce)

Screenshot Placeholder
(Product Search Results Page)

Objectives

- Relevance
- Revenue / margin
- Customer satisfaction

Features

- Text match (BM25/embeddings)
- Price + discounts

Feedback

- Clicks, add-to-cart
- Purchases/returns
- Position bias

Scenario 2: Academic Search (Scholar)

Screenshot Placeholder
(Google Scholar Results Page)

Objectives

- Relevance
- Authority / impact
- Recency (sometimes)

Features

- Text similarity
- Citations, PageRank-like

Feedback

- Clicks / saves
- Citations (delayed)
- Popularity bias

Scenario 3: Streaming (Netflix / Spotify)

Screenshot Placeholder
(Streaming Home Screen)

Objectives

- Engagement (watch time)
- Retention (long-term)

Features

- User/item embeddings
- Recent interactions
- Context (time/device)

Feedback

- Plays, skips
- Watch completion
- Dwell time is noisy

Scenario 4: Online Advertising

Screenshot Placeholder
(Search Page with Ads)

Objectives

- Revenue
- User experience
- Advertiser value

Features

- Bid, predicted CTR
- Ad quality score
- Query-ad relevance

Feedback

- Clicks, conversions
- Attribution noise
- Strategic bidding

What Is Common Across All These Use Cases?

We keep seeing the same pattern.

Large feature space

- 100s–1000s signals
- strong interactions

Multi-objective

- relevance + business
- long-term vs short-term

Noisy feedback

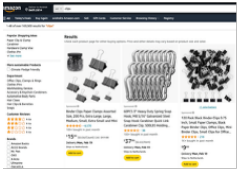
- clicks are biased
- delayed outcomes

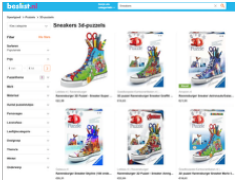
So how do we learn a good ranking function?

Relevance is Multi-faceted

Industrial search engines must optimize for more than just "topical" relevance.

- **Textual**: BM25, semantic match.
- **Quality**: Ratings, reviews.
- **Business**: Price, Prime shipping, revenue per click.





The Scale of Industrial Ranking

Modern search giants handle a massive explosion of signals.

- **Scope**: Amazon, eBay, and Google use **1000+ features** per document.
- **The Wall**: No human can manually tune 1000 weights $\mathbf{w}$ to optimize a metric like NDCG.
- **Heterogeneity**: Combining BM25 (float) with "Is Fresh" (binary) and "Price" (currency).

The LTR Necessity

We need a principled, automated way to learn the optimal "recipe" for combining these signals.

A Document as a Feature Vector

Learning to Rank represents each query-document pair as a vector x.

$$\Phi(q, d) = \underbrace{\begin{array}{c|c|c|c|c} x_1 & x_2 & x_3 & \cdots & x_n \\ \text{BM25 Score} & \text{PageRank} & \text{Click Count} & & \text{Freshness} \end{array}}_{\mathbf{x}_{q,d} \in \mathbb{R}^{1000+}}$$

The LTR Goal

Learn $f(\mathbf{x}) \rightarrow$ score for optimal ranking

Why Learning to Rank?

Problem: We need a scoring function that combines many signals:

$$s(q, d) = f(x_{q,d})$$

where $x_{q,d}$ can have hundreds of components.

- Hand-tuned rules do not scale (too many interactions).
- Objectives conflict (relevance vs revenue vs engagement).
- Supervision is weak and biased (implicit feedback).

Learning to Rank is the engineering solution.

Where LTR Fits: The Multi-Stage Pipeline

LTR is usually the final "re-ranking" stage in a complete stack.

Stage 1: Sparse Retrieval (L3) *or* Dense Retrieval (L5)

↓

Optional: Query Expansion / PRF improves recall (L3)

↓

Stage 2: Neural Re-ranking / Cross-Encoder

↓

Stage 3: Learning to Rank (LTR) combines all signals (L6) (Today)

↓

Final Ranked List → User or RAG Application

Recap: Why LTR is Necessary

Our journey moved from hand-crafted matchers to complex neural-classical ensembles.

- **L3–L4**: BM25 and BERT Cross-Encoders (Standard 2-stage pipeline).
- **L5**: Dense retrieval with Bi-Encoders and Late Interaction (ColBERT).
- **L5 (cont)**: Generative retrieval (DSI, SEAL).

The Unasked Question

How do we **train** these components together to optimize for the final ranking metrics we actually care about (NDCG, MAP)?

Today's Goal: Directly Optimizing Rankings

The core problem is learning a scoring function that respects document ordering.

1. **Pointwise**: Treat relevance as independent regression/classification.
2. **Pairwise**: Learn relative preferences between pairs of documents.
3. **Listwise**: Optimize the entire ranked permutation at once.

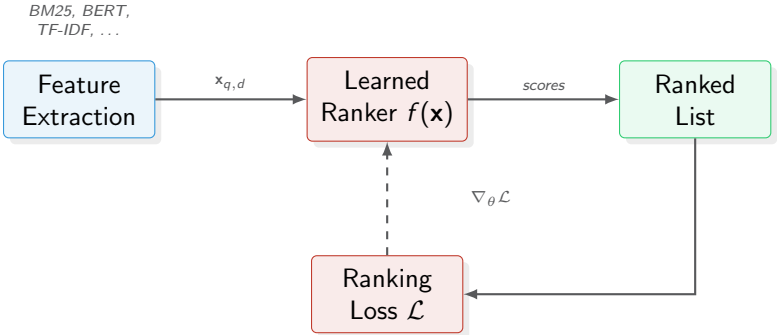
The Non-Differentiable Barrier

Ranking metrics depend on **sorting**, which is discrete. We must find differentiable surrogates or smooth approximations to use Gradient Descent.

The Learning to Rank Pipeline

LTR separates feature extraction from the learned scoring and optimization logic.

- **Traditional**: Hand-crafted features (BM25, PageRank) + LambdaMART.
- **Neural**: End-to-end representation learning + Ranking loss.



Section 3

Evaluation Metrics — What Are We Optimising?

Recap: Binary Metrics

Traditional metrics focus on whether a document is "relevant" or not.

- **P@k / R@k**: Good for broad recall but position-blind within top-k.
- **MRR**: Excellent for navigational/QA tasks (cares about first result).
- **MAP**: Averages precision across all relevant docs; sensitive to rank.

The Limitation

Binary metrics ignore the **intensity** of relevance. In modern search, we often have graded labels (e.g., Perfect, Good, Fair, Bad).

NDCG: The Gold Standard for LTR

NDCG handles graded relevance and prioritizes top-heavy rankings.

- **Grades**: Cumulative Gain (CG) sums relevance scores: $\sum \text{rel}_i$.
- **Discount**: DCG applies a logarithmic decay: $\sum \frac{\text{rel}_i}{\log_2(i+1)}$.
- **Normalization**: NDCG scales DCG to $[0, 1]$ relative to the ideal sort.

$$\text{NDCG@}k = \frac{\sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}}{\text{IDCG@}k}$$

Worked Example: Computing NDCG@5

Let's calculate the cost of a "miss" in the top positions.

- **Current**: Grades $[2, 3, 1, 0, 0] \rightarrow \text{DCG} \approx 4.39$.
- **Ideal**: Grades $[3, 2, 1, 0, 0] \rightarrow \text{DCG} \approx 4.76$.
- **Result**: $\text{NDCG} = 4.39/4.76 \approx \mathbf{0.92}$.

The Logarithmic Effect

Notice that moving the "3" from position 1 to 2 caused an 8% drop in score. NDCG is extremely sensitive to errors at the very top.

The Main Challenge for LTR

The sorting function (or the rank of a document) is difficult to optimize directly.

Ranking metrics (NDCG, MAP, etc.) as functions of document scores are:

- **Non-smooth**
- **Mostly flat**
- **Discontinuous**

The Derivative Problem

Small score changes $\Rightarrow$ no metric change.
Tiny score swap $\Rightarrow$ big metric jump.

Gradients of the true metric are unusable. Each family of LTR methods has its own approach to overcoming this problem.

Categorizing LTR methods?

Most existing work describes three categories: **Pointwise, Pairwise and Listwise**.

Despite wide usage:

- There is not a clear consistently-used definition of each.
- Some methods are categorized differently by different works.
- It is not useful to see them as strictly non-overlapping categories.

A Better Perspective

Think in terms of **properties** instead of categories.

Section 4

Pointwise Approaches

1. Where Do Labels Come From?

To train a model, we need target labels y for query-document pairs.

- **Explicit Ratings**: Human judges grade relevance (e.g., 0=Bad to 4=Perfect). Expensive but high quality.
- **Implicit Feedback**: User behavior parsed from logs.
  - **Clicks**: Did the user click? (Binary)
  - **Dwell Time**: How long did they stay?
  - **Skips**: Did they scroll past it?

2. The Training Table

We flatten the data into a standard machine learning dataset.

Each row is an isolated instance: a Query q, a Document d, their extracted features $\mathbf{x}$, and the target label y.

Query ID	Doc ID	BM25 (x_1)	PageRank (x_2)	...	Label (y)
q_1	d_A	12.4	0.85	...	1 (Click)
q_1	d_B	8.1	0.92	...	0 (Skip)
q_1	d_C	15.2	0.12	...	1 (Click)
q_2	d_X	5.5	0.44	...	0 (Skip)

Key Insight

The model sees rows independently. It does not know that d_A, d_B, d_C belong to the same query q_1 during the forward pass.

3. Mapping Features to a Score

A Pointwise model $f(\mathbf{x})$ maps the feature vector to a predicted score $\hat{y}$.

- **Linear Model**: $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$. (Fast, but assumes linear relationships).
- **Neural Network**: $\hat{y} = \text{MLP}(\mathbf{x})$. (Captures complex feature interactions).
- **Tree Ensemble**: $\hat{y} = \sum \text{Tree}(\mathbf{x})$. (Handles heterogeneous features well).

4. The Pointwise Loss per Example

We train the model to minimize the error between prediction $\hat{y}$ and label y.

- **Regression** (e.g., MSE): $\mathcal{L} = (y - \hat{y})^2$
- **Classification** (e.g., BCE): $\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$

What is it actually doing?

For q_1, d_A, the loss penalizes the model if $\hat{y}$ is far from 1. It updates the weights θ to make $\hat{y}$ closer to 1 **in isolation**.

Limitation of Pointwise Methods

Pointwise losses do not directly optimize ranking quality. A lower loss does not mean a better ranking!

Model A

Rel.	Score	Rank
1	0.6	1
0	0.5	2
0	0.5	3
0	0.5	4
0	0.5	5

Model B

Rel.	Score	Rank
1	0.1	5
0	0.2	1
0	0.2	2
0	0.2	3
0	0.2	4

Mismatch: Ranking is fundamentally about the **relative order** of documents, not fitting a curve.

Pointwise in Disguise: Standard Cross-Encoders

Recall monoBERT from Lecture 4 is technically a pointwise ranker.

- **Input**: Joint query-document sequence `[CLS] Q [SEP] D`.
- **Loss**: Binary Cross-Entropy on the `[CLS]` representation.
- **Success**: Works well because BERT captures deep semantics, but **not** because the loss is optimal for ranking.

The Capacity Paradox

With enough data and model capacity (like LLMs), simplistic pointwise losses often reach surprisingly high performance.

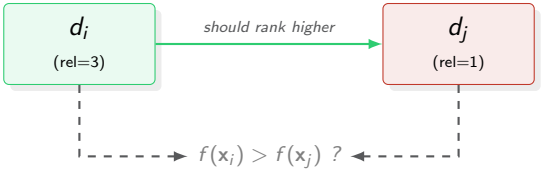
Section 5

Pairwise Approaches

Pairwise: Relative Preferences

Instead of absolute scores, we learn which document is "better" in a pair.

- **Intuition**: For each pair (d_i, d_j), if $y_i > y_j$, the model should score $s_i > s_j$.
- **Strength**: Transforms ranking into binary classification of pairs.
- **Feedback**: Errors are symmetric — push one up, the other down.



Goal: Maximize agreement with labels

RankNet: Learning from Pairs

Pairwise loss minimizes the average number of pair-inversions in rankings.

For all pairs where $y_i > y_j$:

$$\mathcal{L}_{pair} = \sum_{y_i > y_j} \phi(s_i - s_j)$$

RankNet is the most prominent pairwise method:

$$\mathcal{L}_{RankNet} = \sum_{y_i > y_j} \log(1 + e^{-(s_i - s_j)})$$

Burges, C. et al. Learning to rank using gradient descent. ICML 2005.

Limitation of Pairwise Methods

We minimize incorrect inversions. However, not every pair is equally important.

Ranking 1

Rel.	Score	Rank
1	0.6	1
0	0.2	4
1	0.1	5
0	0.4	3
0	0.8	2

Pairs correct: 9

Ranking 2

Rel.	Score	Rank
0	0.8	1
0	0.6	2
1	0.4	3
0	0.2	4
1	0.1	5

Pairs correct: 10

Mismatch: It's much more important to get the correct ordering of top documents than bottom ones. Pairwise treats all pairs equally.

Worked Example: RankNet Misstep

Suppose a Blog (rel=1) is scored at 2.5, while a Tier-1 list (rel=3) is at 2.1.

- **Conflict**: $P(\text{Tier-1} \succ \text{Blog}) \approx 0.40$ (Model is wrong).
- **Gradient**: Pushes Tier-1 score up ($+0.60$) and Blog down (-0.60).
- **Observation**: RankNet weights all swaps equally, regardless of position.

The Position Blindness

Swapping docs at positions 1–2 should be "more important" than positions 99–100. RankNet's loss doesn't know about positions.

RankSVM: Margin-Based Ranking

Historical significance: First to use implicit feedback (clicks) for LTR.

- **Margin**: Force $\mathbf{w}^\top(\mathbf{x}_i - \mathbf{x}_j) \geq 1$ for correctly ordered pairs.
- **Feed**: Users clicking result i after skipping $j \implies d_i \succ d_j$.
- **Legacy**: Solid theory (max-margin) but less scalable than boosting.

Key Contribution

Joachims (2002) showed that even noisy implicit signals (clicks) can train a competitive ranker when framed as a pairwise problem.

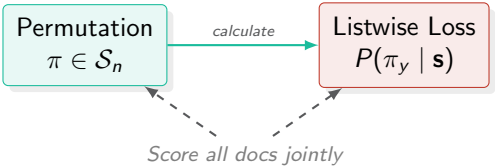
Section 6

Listwise Approaches

Listwise: Optimising the Distribution

Listwise models treat the entire set of documents as a single unit.

- **Philosophy**: Don't look at doc pairs; look at the *permutation*.
- **Models**: $P(\text{ranking} \mid \text{scores})$ is modelled as a probability distribution.
- **Loss**: Minimize KL-divergence between predicted and ideal distributions.



Goal: Minimize distance between predicted and ideal permutations

Connection: Knowledge Distillation as LTR

Teaching a Bi-Encoder (Student) to mimic a Cross-Encoder (Teacher) is a ranking task.

- **Teacher**: Provides graded relevance scores for top-k documents.
- **Student**: Must learn to reproduce the relative ordering, not just absolute scores.
- **Losses**: Listwise KL-divergence over softmaxes works best.

TAS-B Result

Using pairwise/listwise distillation significantly outperforms simple MSE on the MS MARCO benchmark.

Listwise LTR Methods (Plackett-Luce)

Listwise methods consider the entire ranking in their loss.

The first listwise method (ListNet) optimizes a Soft-Max **Plackett-Luce** distribution. Ranking is formulated as sequential sampling without replacement:

$$P(\pi|s) = \prod_{k=1}^{n} \frac{e^{s_{\pi_k}}}{\sum_{j=k}^{n} e^{s_{\pi_j}}}$$

Loss minimizes the cross-entropy with the "best possible" ranking.

Cao, Z. et al. Learning to rank: from pairwise... to listwise approach. ICML 2007.

Limitation of Early Listwise Methods

ListNet assumes there is a single best ranking and optimizes its likelihood.

The Metric Disconnect

Likelihood maximization **does not directly optimize** NDCG or MAP.

Improved likelihood $\neq$ improved evaluation metric.
There is no actual ranking metric involved in original Listwise methods.

Section 7

Metric-Based Methods & LambdaMART

Metric-Based LTR Methods (LambdaRank)

Metric-based LTR aims to optimize a specific ranking metric directly.

Almost all ranking metrics are listwise, thus metric-based implies listwise.
The most-influential metric-based method is **LambdaRank**. It takes the Pairwise loss of RankNet, and weights each pair by how much swapping them affects the metric (e.g., $|\Delta\text{NDCG}|$):

$$\mathcal{L}_{\text{LambdaRank}} = \sum_{y_i > y_j} \log(1 + e^{-(s_i - s_j)}) \cdot |\Delta\text{NDCG}|$$

The gradients treat these weights as constants. Focus is on errors at the top!

Burges, C. J. et al. Learning to rank with nonsmooth cost functions. NIPS 2006.

The Lambda Insight

LambdaRank defines gradients directly, without an explicit loss function.

- **Problem**: NDCG is non-differentiable (jumpy).
- **Solution**: Specify the physical "force" (λ) acting on each doc.
- **Analogy**: "Pull" relevant docs up, "push" irrelevant ones down.

A Theory Mystery

Surprisingly, optimizing these gradients leads to a local optimum for the target ranking metric (Burges et al., 2006).

Worked Example: Lambda Power

Correcting errors at the top yields much higher "gravitational" force.

- **Top Swap** (Pos 1–2): Resulting $|\Delta \text{NDCG}| = \mathbf{0.154}$.
- **Bottom Swap** (Pos 3–4): Resulting $|\Delta \text{NDCG}| = \mathbf{0.030}$.
- **Impact**: The top swap exerts $5\times$ more gradient force on the model than the bottom swap.

Convergence

This weighting causes the model to "give up" on tail documents and focus exclusively on perfecting the top-10 results.

LambdaMART: Industry Workhorse

Combines Lambda gradients with Gradient Boosted Regression Trees.

- **MART**: Multiple Additive Regression Trees (Boosting).
- **Process**: Compute current λ for all docs $\rightarrow$ Fit a tree to predict those λ values $\rightarrow$ Add tree to ensemble.
- **Standard**: Powering Bing, Yahoo, and specialized e-commerce search for years.

Why Trees?

Non-neural trees handle **heterogeneous features** (BM25 score + PageRank + Category ID) much better than MLPs without massive feature engineering.

LTR Features: What the Model "Sees"

Traditional LTR models use a large bank of query-document features.

- **Lexical**: BM25, TF-IDF, Language Model scores.
- **Structural**: In-link count (PageRank), URL length, title matches.
- **Modern**: Dense similarity (L5), Cross-encoder scores (L4).

The Ensemble Effect

LambdaMART doesn't replace BM25; it uses BM25 as **one of many inputs**. It learns the optimal "mixture" of classical and semantic signals.

Rank Metric Approximation

ApproxDCG approximates the rank function through a sigmoid upper bound.

$$\hat{\text{rank}}_i = 1 + \sum_{j \neq i} \sigma(s_j - s_i)$$

This differentiable approximate rank is then plugged into the DCG objective:

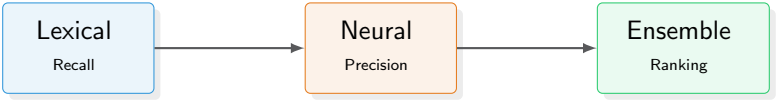
$$\text{ApproxDCG} = \sum_{i=1}^{n} \frac{2^{y_i} - 1}{\log_2(1 + \hat{\text{rank}}_i)}$$

Qin, T. et al. A general approximation framework for direct optimization... IRJ 2010.

The Modern Retrieve-Then-Rerank Stack

Modern search engines are multi-stage ensembles of different LTR paradigms.

- **Stage 1**: Dense/Sparse Retrieval (Recall optimized, Pointwise InfoNCE).
- **Stage 2**: Neural Re-ranker (High interaction, Pairwise BERT).
- **Stage 3**: LambdaMART (Aggregates all signals + metadata).



Practical LTR: LightGBM & PyTerrier

Standardized libraries make it easy to train metric-aware rankers.

- **LightGBM**: The primary engine for LambdaMART in industrial search.
- **PyTerrier**: Orchestrates the pipeline from indexing to ensemble training.
- **Objective**: Simply set `"objective": "lambdarank"` to start.

Feature BANK

The power of LambdaMART comes from its ability to use **50+ features**: BM25, PageRank, BERT scores, and URL statistics.

Stochastic LTR Methods

Inspired by reinforcement learning, optimize the expected utility of a probabilistic ranking model:

$$\mathbb{E}_{R\sim\pi_\theta}[\text{Metric}(R)]$$

The number of possible rankings is enormous, so it is only feasible to approximate this expected value via **sampling**.

The REINFORCE log-derivative trick approximates the gradient from samples:

$$\nabla_\theta \mathbb{E}_{R\sim\pi_\theta}[\text{Metric}(R)] = \mathbb{E}_{R\sim\pi_\theta}[\text{Metric}(R)\nabla_\theta \log P(R|\theta)]$$

Oosterhuis, H. Computationally efficient optimization of Plackett-Luce ranking... SIGIR 2021.

Big Picture Comparison

Method	What It Optimizes
Pointwise	Regression/classification loss
Pairwise	Pair inversions
Listwise	Ranking likelihood
Metric-based	Metric-informed gradients
Stochastic	Expected metric

Section 9

Beyond Standard LTR: Counterfactual & Online

The Problem with Clicks: Position Bias

Implicit feedback (clicks) is cheap and abundant, but heavily biased.

- **Position Bias**: Users are much more likely to click the top result simply because it is at the top, *even if it's less relevant* than the result at position 3.
- **The Echo Chamber**: If we train LTR directly on raw clicks, the model learns to reproduce the existing biased ranking!
- **Trust Bias**: Users trust the search engine and might click a bad top result before realizing it's useless (hence, "skips" and "dwell time" help, but don't cure the bias).

The Counterfactual Question

"Would the user have clicked Document B if it had been shown at Position 1 instead of Document A?"

Counterfactual LTR (CLTR): Inverse Propensity Scoring

CLTR attempts to un-bias the click data during training.

- **Propensity (p_k)**: The probability a user *examines* position k. ($p_1 = 1.0, p_3 = 0.4$).
- **IPS Weighting**: Up-weight clicks that happen at lower, less-examined positions.
- **Example**: A click at position 3 ($p = 0.4$) is worth $1/0.4 = \mathbf{2.5}\times$ more heavily in the loss than a click at position 1.

Placeholder Image

(e.g., Eye-tracking heatmap showing high attention on top)

The Result

The Problem with Offline LTR: Static Datasets

Training on historical Data (X, Y) assumes the world stands still.

- **Concept Drift**: What is relevant for "apple" today (iPhone 15) is different from what was relevant 6 months ago (iPhone 14).
- **Cold Start**: New documents have no historical click data or explicit ratings.
- **Expensive Updates**: Re-training a massive LambdaMART ensemble and deploying it takes days or weeks.

The Static Limitation

Standard LTR models degrade over time because they cannot adapt immediately to breaking news, viral trends, or shifting user intents.

Online LTR (OLTR): Learning from Live Interactions

OLTR models learn directly and immediately by interacting with users.

- **Exploration vs. Exploitation**: We must safely test new ranking strategies without ruining the user experience.
- **Interleaving**: Combine the results of Ranker A (current) and Ranker B (new) by alternating them.
- **Dueling Bandits (DBGD)**: Whichever ranker gets more clicks in the interleaved list "wins", and the model updates in that direction immediately.

Placeholder Image

(e.g., Team Draft Interleaving Ranker A vs Ranker B)

Section 10

Summary: Choosing Your Paradigm

Summary: Choosing Your Paradigm

The trade-off is between ranking awareness and computational complexity.

- **Pointwise**: Fast, scalable, but position-blind. Best for 1st stage.
- **Pairwise**: The standard for re-ranking. LambdaMART is the workhorse.
- **Listwise**: Use for deep semantic interaction or small top-k re-ranking.

The Recall Ceiling

Remember: A perfect re-ranker cannot fix what the first stage missed. LTR is most effective when the retrieval stage has high recall.

Next Class: Evaluation & Large Scale Systems

Next week, we move into the "wild" — how to evaluate systems with real users.

- **Offline**: Benchmarks like TREC and the limits of static labels.
- **Online**: A/B testing, Interleaving, and Click-through analysis.
- **Deployment**: Building low-latency production pipelines.

Preparation

Review the PyTerrier documentation for the Learning to Rank module.